Git & GitHub

Jose Tupayachi

Department of Industrial & Systems Engineering — UTK

March 18, 2026

THE UNIVERSITY OF TENNESSEE KNOXVILLE

Git & GitHub

Jose Tupayachi

Department of Industrial & Systems Engineering — UTK

March 18, 2026

THE UNIVERSITY OF TENNESSEE KNOXVILLE

- Today we'll talk about Git and GitHub
- Essential tools for version control and collaboration
- Used in most modern software and research projects

Agenda

Part	Topic
1	Setup & Installation
2	Git Basics — Your First Repo
3	Working with GitHub Remotes
4	Branching & Merging
5	Collaboration — Pull Requests & Code Review
6	Best Practices

*We'll cover both the **command line** and **GitHub Desktop** (GUI) for every workflow, with live demos throughout.*

Agenda

Part	Topic
1	Setup & Installation
2	Git Basics — Your First Repo
3	Working with GitHub Remotes
4	Branching & Merging
5	Collaboration — Pull Requests & Code Review
6	Best Practices

*We'll cover both the **command line** and **GitHub Desktop** (GUI) for every workflow, with live demos throughout.*

- first of all ill be covering the setup and installation
- then we will cover git basics — creating a repo, making commits, viewing history, undoing changes, and using .gitignore
- next we will connect our local repo to GitHub, push and pull changes, and discuss the difference between cloning, forking, and creating cloud repos
- after that we will dive into branching and merging — creating branches, working on them, and merging back to main
- then we will cover collaboration workflows — how to contribute to other people's projects using pull requests and code reviews
- finally we will wrap up with some best practices for using Git in your day-to-day projects

Why Version Control?

The Problem (sound familiar?):

- `simulation_model_final.py`
- `simulation_model_final_v2.py`
- `simulation_model_FINAL_really_final.py`
- "Which version of the optimization script has the latest constraints?"
- "Who changed the demand forecast parameters and when?"
- "I broke the scheduling model — how do I go back?"

The Solution: Git

- Track every change to your scripts, data pipelines, and reports
- Revert to any previous state instantly
- Collaborate on team projects without overwriting each other's work

Why Version Control?
The Problem (sound familiar?):
simulation_model_final.py
simulation_model_final_v2.py
simulation_model_FINAL_really_final.py
"Which version of the optimization script has the latest constraints?"
"Who changed the demand forecast parameters and when?"
"I broke the scheduling model — how do I go back?"
The Solution: Git
Track every change to your scripts, data pipelines, and reports
Revert to any previous state instantly
Collaborate on team projects without overwriting each other's work

- Many file versions — hard to manage
- Not clear which is the latest
- Hard to track who changed what
- Hard to undo mistakes
- Git tracks line-by-line changes
- We can go back to any older version instantly
- Teamwork becomes much easier

Git vs. GitHub

Git	GitHub
Version control tool	Cloud hosting platform
Runs locally	Accessible from anywhere
Command-line based	Web interface + API
Free & open source	Free tier + paid plans

Analogy

Git is like a camera (takes snapshots).
GitHub is like a photo album in the cloud (stores & shares them).

Git vs. GitHub
Analogy
Git is like a camera (takes snapshots).
GitHub is like a photo album in the cloud (stores & shares them).

- Git and GitHub are not the same thing
- Git is the version control system that runs locally on your computer
- GitHub is a cloud platform that hosts Git repositories
- Other platforms: GitLab, Bitbucket, Azure DevOps — same Git commands, different hosts

Prerequisites: Creating a GitHub Account

1. Go to `https://github.com`
2. Click **Sign Up**
3. Choose a username, enter your email, create a password
4. Verify your email address

Prerequisites: Creating a GitHub Account
1. Go to https://github.com
2. Click Sign Up
3. Choose a username, enter your email, create a password
4. Verify your email address

- Everyone needs a GitHub account to follow along
- Go to github.com and sign up if you don't have one
- It's free and only takes a minute

Part 1: Setup & Installation

Setup & Installation

Part 1: Setup & Installation
Setup & Installation

1.1 Installing Git (command line)

macOS (usually pre-installed):

```
git --version
# or install via Homebrew:
brew install git
```

Windows:

- Download from `https://git-scm.com/download/win`

Linux (Ubuntu/Debian):

```
sudo apt update && sudo apt install git
```


1.1 Installing Git (command line)
macOS (usually pre-installed):
Windows:
Download from https://git-scm.com/download/win
Linux (Ubuntu/Debian):

- macOS: install Homebrew first from `https://brew.sh`
- Windows: open PowerShell with admin rights before installing
- Windows: install Chocolatey from `https://chocolatey.org/install`

1.2 Configuring Git

Set your identity (attached to every commit):

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Set default branch name:

```
git config --global init.defaultBranch main
```

Verify:

```
git config --list
```


1.2 Configuring Git
Set your identity (attached to every commit):
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
Set default branch name:
git config --global init.defaultBranch main
Verify:
git config --list

1.4 SSH Key Authentication (Recommended)

Step 1 — Generate a key:

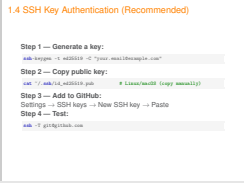
```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

Step 2 — Copy public key:

```
cat ~/.ssh/id_ed25519.pub          # Linux/macOS (copy manually)
```

Step 3 — Add to GitHub:
Settings → SSH keys → New SSH key → Paste
Step 4 — Test:

```
ssh -T git@github.com
```

- Step 3b (Rarely needed): Start the SSH agent and add the key
- `eval "$(ssh-agent -s)"`
- `ssh-add ~/.ssh/id_ed25519`

1.5 GitHub Desktop (Desktop GUI)

Install GitHub Desktop now: `https://desktop.github.com`

- Sign in with your GitHub account
- It auto-configures your Git identity

1.5 GitHub Desktop (Desktop GUI)
Install GitHub Desktop now: https://desktop.github.com
Sign in with your GitHub account
It auto-configures your Git identity

- GitHub Desktop simplifies Git with a visual interface
- No need to manually configure remotes or stage files from the terminal
- Use buttons and checkboxes instead of typing commands

Exercise 1

1. Verify Git is installed: `git - -version`
2. Configure your name and email with `git config`
3. Create a GitHub account (if you don't have one)
4. **Install GitHub Desktop** from `https://desktop.github.com`
5. Set up SSH key authentication (not needed for GitHub Desktop)
6. Test with `ssh -T git@github.com`

Live Demo

I'll walk through this setup on my machine right now.

Exercise 1

1. Verify Git is installed: `git - -version`
2. Configure your name and email with `git config`
3. Create a GitHub account (if you don't have one)
4. **Install GitHub Desktop** from `https://desktop.github.com`
5. Set up SSH key authentication (not needed for GitHub Desktop)
6. Test with `ssh -T git@github.com`

Live Demo

I'll walk through this setup on my machine right now.

- GitHub Desktop emulates all the same functionality as the terminal
- Great for visualizing changes before committing

Git Repos — First Repository

THE UNIVERSITY OF TENNESSEE KNOXVILLE

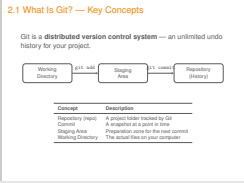
Part 2: Git Repos

Git Repos — First Repository

2.1 What Is Git? — Key Concepts

Git is a **distributed version control system** — an unlimited undo history for your project.

Concept	Description
Repository (repo)	A project folder tracked by Git
Commit	A snapshot at a point in time
Staging Area	Preparation zone for the next commit
Working Directory	The actual files on your computer



2.2 Creating Your First Repository

Command Line:

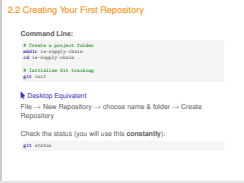
```
# Create a project folder
mkdir ie-supply-chain
cd ie-supply-chain

# Initialize Git tracking
git init
```

Desktop Equivalent

File → New Repository → choose name & folder → Create Repository

Check the status (you will use this **constantly**):

```
git status
```

- Run `ls -a` to show the hidden `.git` folder
- Check status with `git status`
- Also show how to import the repo in GitHub Desktop

2.2a Creating a Repo from GitHub Desktop

Step-by-step in GitHub Desktop:

1. Open GitHub Desktop
2. Go to **File** → **New Repository…**
3. Fill in:
   - **Name:** `ie-supply-chain`
   - **Local Path:** choose where to save it
   - (Optional) check **Initialize this repository with a README**
   - (Optional) pick a **.gitignore** template (e.g. Python)
4. Click **Create Repository**

GitHub Desktop creates the folder, runs `git init`, and opens the repo — no terminal needed.

To open the folder in your editor:
Repository → Open in *Visual Studio Code* (or your default editor)

2.2a Creating a Repo from GitHub Desktop
Step-by-step in GitHub Desktop:
Open GitHub Desktop
Go to File → New Repository…
Fill in:
Name: ie-supply-chain
Local Path: choose where to save it
(Optional) check Initialize this repository with a README
(Optional) pick a .gitignore template (e.g. Python)
Click Create Repository
GitHub Desktop creates the folder, runs git init, and opens the repo — no terminal needed.
To open the folder in your editor:
Repository → Open in Visual Studio Code (or your default editor)

- Point out the **Current Repository** dropdown in the top-left — that's how you switch between repos
- The **No local changes** message confirms the repo is clean right after creation
- Show the **History** tab — it already has one commit if README was checked

2.2b Creating a Cloud Repo & Cloning Locally

Step 1 — Create the repo on GitHub:

1. Go to `https://github.com/new`
2. Enter a name, choose Public or Private
3. Check **Add a README file** (and optionally a .gitignore)
4. Click **Create repository**

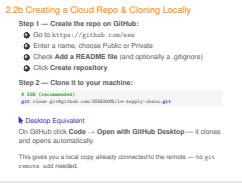
Step 2 — Clone it to your machine:

```
# SSH (recommended)
git clone git@github.com:USERNAME/ie-supply-chain.git
```

Desktop Equivalent

On GitHub click **Code** → **Open with GitHub Desktop** — it clones and opens automatically.

This gives you a local copy *already connected* to the remote — no `git remote add` needed.


2.2b Creating a Cloud Repo & Cloning Locally
Step 1 — Create the repo on GitHub:
Go to https://github.com/new
Enter a name, choose Public or Private
Check Add a README file (and optionally a .gitignore)
Click Create repository
Step 2 — Clone it to your machine:
SSH (recommended)
git clone git@github.com:USERNAME/ie-supply-chain.git
Desktop Equivalent
On GitHub click Code → Open with GitHub Desktop — it clones and opens automatically.
This gives you a local copy already connected to the remote — no git remote add needed.

- This is the **easiest starting point** for new projects — create on GitHub first, then clone
- The clone command sets up the `origin` remote automatically
- Difference from 2.2: here GitHub is the source of truth; in 2.2 the local folder is
- Show the **Code** button on GitHub and the clone URL options (SSH vs HTTPS)

2.3 The Git Workflow: Add → Commit

`not staged | not committed`

Command Line:

```
# Step 1: Create a file
echo "# Supply Chain Analysis" > README.md

# Step 2: Stage the file
git add README.md

# Step 3: Commit
git commit -m "Initial commit: add README"
```

Desktop Equivalent

Changed files appear in the left panel. Check the boxes to stage, type a commit message at the bottom, click **Commit to main**.

Live Demo

GitHub Desktop side by side add and commit

2.3 The Git Workflow: Add → Commit
not staged | not committed
Command Line:
Step 1: Create a file
echo "# Supply Chain Analysis" > README.md
Step 2: Stage the file
git add README.md
Step 3: Commit
git commit -m "Initial commit: add README"
Desktop Equivalent
Changed files appear in the left panel. Check the boxes to stage, type a commit message at the bottom, click Commit to main.
Live Demo
GitHub Desktop side by side add and commit

- Install VS Code on Windows: `choco install vscode`
- Install Python 3 on Windows: `choco install python310`
- After installing Python on Windows, disable “App execution aliases” in Settings
- Windows: use `python`; Linux/macOS: use `python3`
- also we can load the repo to github desktop we can link it
- for the live demo "desktop" we can create a new repo from github desktop too! Create a new repo

2.4 Making Changes & Viewing Diffs

Command Line:

```
# Edit the file
echo "## About" >> README.md
echo "Supply chain optimization project." >> README.md

# See what changed
git diff            # exact line-by-line changes (before adding
     changes)
```

`git diff` output: lines with + were **added**, lines with - were **removed**.

```
git add README.md
git commit -m "Add About section to README"
```

Desktop Equivalent

Click any changed file to see a **visual diff** (green/red highlighting). No commands needed!

2.4 Making Changes & Viewing Diffs
Command Line:
Edit the file
echo "## About" >> README.md
echo "Supply chain optimization project." >> README.md
See what changed
git diff # exact line-by-line changes (before adding changes)
git diff output: lines with + were added, lines with - were removed.
git add README.md
git commit -m "Add About section to README"
Desktop Equivalent
Click any changed file to see a visual diff (green/red highlighting). No commands needed!

2.5 Viewing History

Command Line:

```
git log                          # full commit log
git log --oneline                # compact view
git log --oneline --graph --all # graphical view
git show <commit-hash>           # specific commit
```

Desktop Equivalent

Click the **History** tab to see all commits with diffs, authors, and timestamps in a visual timeline.

2.5 Viewing History
Command Line:
git log # full commit log
git log --oneline # compact view
git log --oneline --graph --all # graphical view
git show <commit-hash> # specific commit
Desktop Equivalent
Click the History tab to see all commits with diffs, authors, and timestamps in a visual timeline.

2.6 Undoing Things

```
# You staged "added ." a file but changed your mind -- unstage it
    (keep your edits take it out of the added files)
git restore --staged analysis.py

# You edited a file and want to throw away all changes (go back to
    last commit, or simply remove changes for that file)
git restore analysis.py              # WARNING: permanent, edits are
    gone! What was modified will be removed

# You just committed but the message has a typo
git commit --amend -m "feat: add demand forecast (corrected)" # if
    nothing added it will just change the name of what was committed

git add analysis.py
git commit --amend --no-edit       # rewrites last commit to include
    analysis.py, you can use a new message or simply no edit to keep
    the same message
```

Desktop Equivalent

Right-click a file → **Discard Changes**. To amend: check **Amend commit** at the bottom of the commit panel before clicking Commit.

2.6 Undoing Things

You just committed but forgot to include a file you already edited # (only works if you have NOT pushed yet, or else you have to do a new commit and re push)

2.7 The .gitignore File

Tells Git which files to **never track**. Create a file called `.gitignore` in your repo root:

```
# Compiled / generated files
*.pyc
__pycache__/

# OS files
.DS_Store
Thumbs.db

# IDE files
.vscode/
.idea/

# Large data (track with Git LFS instead)
*.csv
*.xlsx
data/raw/

# Secrets & environment
.env
*.key
```


2.7 The .gitignore File
Tells Git which files to never track. Create a file called .gitignore in your repo root:
Compiled / generated files
*.pyc
__pycache__/
OS files
.DS_Store
Thumbs.db
IDE files
.vscode/
.idea/
Large data (track with Git LFS instead)
*.csv
*.xlsx
data/raw/
Secrets & environment
.env
*.key

2.7b .gitignore — Pattern Cheat Sheet

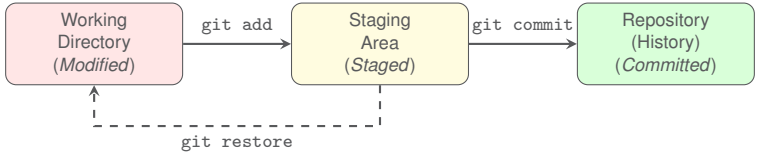
Pattern	Matches	Example
`*.csv`	All CSV everywhere	`sales.csv`
`build/`	Directory (1 hop)	`build/model.pkl`
`data/`	Directory (even innner dirs)	`data/raw/file.txt`
`!data/README.md`	Exception	keep one file
`*.log`	Log files	`run.log`
`**/*.tmp`	Any Temp file	`src/a/b.tmp`

Auto-generate a starter `.gitignore`**:**

```
# Use gitignore.io (now toptal.com)
curl -sL "https://www.toptal.com/developers/gitignore/api/\
python,macos,windows,visualstudiocode" > .gitignore
```

Or browse templates at
`https://github.com/github/gitignore`

2.8 Understanding the Three States



State t_0	Description	Next Command t_1
Modified	Not staged	`git add`
Staged	Ready to commit	`git commit`
Committed	In history	—

2.8 Understanding the Three States

Exercise 2 — Part A

Try both the command line AND GitHub Desktop:

1. Create a new folder `ie-project` and initialize a Git repo
2. Create at least 3 files (e.g., `README.md`, `analysis.py`, `.gitignore`)
3. Make an initial commit with all files
4. Edit `analysis.py` and view the diff
5. Make a second commit
6. View your history with `git log -oneline` (or the History tab)

Live Demo

I'll do this exercise live — terminal on the left, GitHub Desktop on the right.

Exercise 2 — Part A
Try both the command line AND GitHub Desktop:
1. Create a new folder ie-project and initialize a Git repo
2. Create at least 3 files (e.g., README.md, analysis.py, .gitignore)
3. Make an initial commit with all files
4. Edit analysis.py and view the diff
5. Make a second commit
6. View your history with git log -oneline (or the History tab)
Live Demo
I'll do this exercise live — terminal on the left, GitHub Desktop on the right.

Exercise 2 — Part B: Staging & Restore

Practice undoing things (command line):

1. Edit `analysis.py` — then **stage** it with `git add`
2. Change your mind: **unstage** it (keep your edits on disk)

```
git restore --staged analysis.py
```

3. Edit `analysis.py` again — this time **discard** all changes entirely

```
git restore analysis.py     # WARNING: edits are gone!
```

4. Make a commit, then realize the message has a typo — fix it

```
git commit --amend -m "feat: corrected message"
```

5. Make a commit, then add a forgotten file to that same commit

```
git add forgotten.py
git commit --amend --no-edit
```


Exercise 2 — Part B: Staging & Restore
Practice undoing things (command line):
1. Edit analysis.py — then stage it with git add
2. Change your mind: unstage it (keep your edits on disk)
git restore --staged analysis.py
3. Edit analysis.py again — this time discard all changes entirely
git restore analysis.py # WARNING: edits are gone!
4. Make a commit, then realize the message has a typo — fix it
git commit --amend -m "feat: corrected message"
5. Make a commit, then add a forgotten file to that same commit
git add forgotten.py
git commit --amend --no-edit

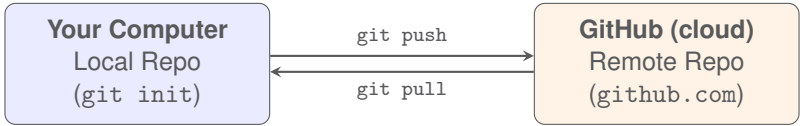
Working with GitHub Remotes

THE UNIVERSITY OF TENNESSEE KNOXVILLE

Part 3: GitHub Remotes

Working with GitHub Remotes

3.1 Local vs. Remote



Other platforms: GitLab, Bitbucket, Azure DevOps — same Git commands, different hosts.

3.1 Local vs. Remote
Other platforms: GitLab, Bitbucket, Azure DevOps — same Git commands, different hosts.

3.2 Creating a Repository on GitHub

Option A: Create on GitHub first:

1. Go to `https://github.com/new`
2. Repository name: `ie-supply-chain`
3. Visibility: Public or Private
4. **Do NOT** initialize with README (we have one locally)
5. Click **Create repository**

Desktop Equivalent

File → New Repository — or Publish Repository if you already have a local repo open.

3.2 Creating a Repository on GitHub

Option A: Create on GitHub first:
1. Go to `https://github.com/new`
2. Repository name: `ie-supply-chain`
3. Visibility: Public or Private
4. **Do NOT** initialize with README (we have one locally)
5. Click **Create repository**

Desktop Equivalent
File → New Repository — or Publish Repository if you already have a local repo open.

3.3 Connecting Local to Remote & Pushing

Add a remote:

```
# SSH (recommended)
git remote add origin git@github.com:USER/ie-supply-chain.git

# HTTPS
git remote add origin https://github.com/USER/ie-supply-chain.git
```

First push:

```
git push -u origin main
```

Desktop Equivalent

Click **Publish Repository** (top bar). Choose public/private. Done — no remote/URL needed!

Live Demo

Publishing our IE project repo — CLI and Desktop.

3.3 Connecting Local to Remote & Pushing
Add a remote:
SSH (recommended)
git remote add origin git@github.com:USER/ie-supply-chain.git
HTTPS
git remote add origin https://github.com/USER/ie-supply-chain.git
First push:
git push -u origin main
Desktop Equivalent
Click Publish Repository (top bar). Choose public/private. Done — no remote/URL needed!
Live Demo
Publishing our IE project repo — CLI and Desktop.

3.4 Cloning a Repository

Command Line:

```
# SSH
git clone git@github.com:USERNAME/REPOSITORY.git

# HTTPS
git clone https://github.com/USERNAME/REPOSITORY.git
```

Desktop Equivalent

File → Clone Repository → paste URL or pick from your GitHub repos list → Clone.

Creates a local copy with full commit history and remote already configured.

3.4 Cloning a Repository
Command Line:
SSH
git clone git@github.com:USERNAME/REPOSITORY.git
HTTPS
git clone https://github.com/USERNAME/REPOSITORY.git
Desktop Equivalent
File → Clone Repository → paste URL or pick from your GitHub repos list → Clone.
Creates a local copy with full commit history and remote already configured.

3.5 Pulling Changes

Command Line:

```
git pull        # fetch + merge in one step
git push        # share your changes
```

Desktop Equivalent

Click **Fetch Origin** (top bar) to check for updates, then **Pull Origin** to download them. Click **Push Origin** to upload your commits.

The Push/Pull Workflow:

1. Make changes locally
2. `git pull` ← get latest first
3. `git add . & git commit -m "description"`
4. `git push` ← share your changes

Always `pull` before you `push` to minimize merge conflicts.

3.5 Pulling Changes
Command Line:
git pull # fetch + merge in one step
git push # share your changes
Desktop Equivalent
Click Fetch Origin (top bar) to check for updates, then Pull Origin to download them. Click Push Origin to upload your commits.
The Push/Pull Workflow:
1. Make changes locally
2. git pull ← get latest first
3. git add . & git commit -m "description"
4. git push ← share your changes
Always pull before you push to minimize merge conflicts.

3.6 Clone vs. Fork vs. Create

Action	When to use	Command
Create	New project	`git init` + `remote add`
Clone	Repo you have access to	`git clone <url>`
Fork	Someone else's project	Fork on GitHub, then clone

3.6 Clone vs. Fork vs. Create

Action	When to use	Command
Create	New project	`git init` + `remote add`
Clone	Repo you have access to	`git clone <url>`
Fork	Someone else's project	Fork on GitHub, then clone

Exercise 3

Pick your tool: Command Line or GitHub Desktop (or both!):

1. Create a new repository on GitHub
2. Connect and push your local repo (CLI: `git remote add` + `push` / Desktop: **Publish Repository**)
3. Verify your files appear on GitHub
4. Edit a file on GitHub directly (pencil icon), commit it
5. Pull the changes locally (CLI: `git pull` / Desktop: **Fetch** then **Pull**)

Live Demo

Pushing and pulling — showing both workflows on the same repo.

Exercise 3

Pick your tool: Command Line or GitHub Desktop (or both!):
1. Create a new repository on GitHub
2. Connect and push your local repo (CLI: git remote add + push / Desktop: Publish Repository)
3. Verify your files appear on GitHub
4. Edit a file on GitHub directly (pencil icon), commit it
5. Pull the changes locally (CLI: git pull / Desktop: Fetch then Pull)

Live Demo
Pushing and pulling — showing both workflows on the same repo.

Part 4: Branching & Merging

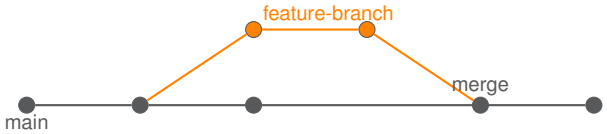
Branching & Merging

Part 4: Branching & Merging

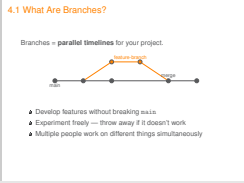
Branching & Merging

4.1 What Are Branches?

Branches = **parallel timelines** for your project.



- Develop features without breaking `main`
- Experiment freely — throw away if it doesn't work
- Multiple people work on different things simultaneously



4.2 Creating & Switching Branches

Command Line:

```
# See current branch
git branch

# Create AND switch in one step
git switch -c feature/demand-forecast
```

Legacy alternative: `git checkout -b feature/demand-forecast`

Desktop Equivalent

Branch → New Branch → type name → Create Branch.
Switching: click the **Current Branch** dropdown and select.

Live Demo

Creating and switching branches in both tools.

4.2 Creating & Switching Branches
Command Line:
See current branch
git branch
Create AND switch in one step
git switch -c feature/demand-forecast
Legacy alternative: git checkout -b feature/demand-forecast
Desktop Equivalent
Branch → New Branch → type name → Create Branch.
Switching: click the Current Branch dropdown and select.
Live Demo
Creating and switching branches in both tools.

4.3 Working on a Branch

Command Line:

```
# 1. Create and switch to a new branch
git switch -c feature/demand-forecast

# 2. Make changes
echo "def forecast_demand(history):" > forecast.py
echo "    return sum(history) / len(history)" >> forecast.py

# 3. Stage and commit
git add forecast.py
git commit -m "feat: add demand forecast function"

# 4. View branch history
git log --oneline --graph --all
```

Desktop Equivalent

Edit files in your editor. Changes auto-appear in Desktop's left panel. Write message → **Commit to feature/demand-forecast**.

4.3 Working on a Branch
Command Line:
Desktop Equivalent
Edit files in your editor. Changes auto-appear in Desktop's left panel. Write message → Commit to feature/demand-forecast.

4.4 Merging Branches

Command Line:

```
# Step 1: Switch to the target branch
git switch main

# Step 2: Merge the feature branch
git merge feature/demand-forecast

# Step 3: Delete the branch (cleanup)
git branch -d feature/demand-forecast
```

Desktop Equivalent

Switch to `main` via the branch dropdown. Then Branch → **Merge into Current Branch** → select the feature branch.

Type	When	Result
Fast-forward	No new commits on `main`	Linear history
Three-way merge	Both branches have commits	Merge commit created

4.4 Merging Branches
Command Line:
Step 1: Switch to the target branch
git switch main
Step 2: Merge the feature branch
git merge feature/demand-forecast
Step 3: Delete the branch (cleanup)
git branch -d feature/demand-forecast
Desktop Equivalent
Switch to main via the branch dropdown. Then Branch → Merge into Current Branch → select the feature branch.
Type When Result
Fast-forward No new commits on main Linear history
Three-way merge Both branches have commits Merge commit created

4.5 Handling Merge Conflicts

Conflicts happen when **two branches modify the same lines**.

What a conflict looks like:

```
<<<<<<< HEAD
Hello from Main
=======
Hello from Branch A
>>>>>>> branch-a
```

Resolution: Edit the file, remove conflict markers, then stage and commit.

VS Code shows “Accept Current”, “Accept Incoming”, “Accept Both” buttons.

4.5 Handling Merge Conflicts

Conflicts happen when **two branches modify the same lines**.

What a conflict looks like:

```
<<<<<<< HEAD
Hello from Main
=======
Hello from Branch A
>>>>>>> branch-a
```

Resolution: Edit the file, remove conflict markers, then stage and commit.

VS Code shows “Accept Current”, “Accept Incoming”, “Accept Both” buttons.

4.6 Branch Naming Conventions

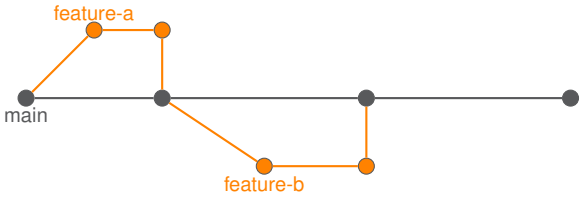
Prefix	Purpose
`feature/`	New functionality
`bugfix/`	Bug fixes
`hotfix/`	Urgent production fixes
`docs/`	Documentation changes
`refactor/`	Code restructuring
`test/`	Adding or modifying tests

Examples:

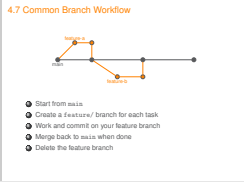
- `feature/demand-forecast`
- `bugfix/fix-inventory-calc`
- `docs/update-process-docs`

4.6 Branch Naming Conventions
Prefix | Purpose
feature/ | New functionality
bugfix/ | Bug fixes
hotfix/ | Urgent production fixes
docs/ | Documentation changes
refactor/ | Code restructuring
test/ | Adding or modifying tests
Examples:
feature/demand-forecast
bugfix/fix-inventory-calc
docs/update-process-docs

4.7 Common Branch Workflow



1. Start from `main`
2. Create a `feature/` branch for each task
3. Work and commit on your feature branch
4. Merge back to `main` when done
5. Delete the feature branch



Exercise 4

Try with both CLI and GitHub Desktop:

1. From `main`, create branch `feature/quality-report`
2. Create `quality_report.py` and make 2 commits
3. Switch to `main`, create `scheduling.py`, commit
4. Merge `feature/quality-report` into `main`
5. View with `git log -oneline -graph -all` (or Desktop History)

Bonus (Conflict Practice):

1. Create two branches that edit the same line in `README.md`
2. Merge the first (no conflict), merge the second (conflict!) and resolve it

Live Demo

Branching, merging, and resolving a conflict — live in both tools.

Exercise 4
Try with both CLI and GitHub Desktop:
1. From main, create branch feature/quality-report
2. Create quality_report.py and make 2 commits
3. Switch to main, create scheduling.py, commit
4. Merge feature/quality-report into main
5. View with git log -oneline -graph -all (or Desktop History)
Bonus (Conflict Practice):
1. Create two branches that edit the same line in README.md
2. Merge the first (no conflict), merge the second (conflict!) and resolve it
Live Demo
Branching, merging, and resolving a conflict — live in both tools.

Pull Requests & Code Review

Part 5: Collaboration

Pull Requests & Code Review

5.1 The Collaboration Workflow

1. Fork or clone the repository
2. Create a feature branch
3. Make changes and push the branch
4. Open a **Pull Request (PR)**
5. Team reviews and discusses
6. Merge the PR into `main`

This is how open-source projects, research labs, and engineering teams work together.

Desktop Equivalent

GitHub Desktop has a **"Create Pull Request"** button that opens GitHub in your browser, pre-filled.

5.1 The Collaboration Workflow

1. Fork or clone the repository
2. Create a feature branch
3. Make changes and push the branch
4. Open a **Pull Request (PR)**
5. Team reviews and discusses
6. Merge the PR into `main`

This is how open-source projects, research labs, and engineering teams work together.

Desktop Equivalent

GitHub Desktop has a **"Create Pull Request"** button that opens GitHub in your browser, pre-filled.

5.2 Pushing a Branch & Opening a PR

Push your feature branch:

```
git switch feature/quality-report
git push -u origin feature/quality-report
```

Desktop Equivalent

Click **Push Origin** (or **Publish Branch** if it's new). Then click **Create Pull Request** — opens GitHub in your browser.

On GitHub:

1. Click **"Compare & pull request"**
2. Fill in: Title, Description, Reviewers, Labels
3. Click **"Create pull request"**

Good PR template:

- **What** — Summary of changes
- **Why** — Motivation/context
- **Testing** — How you verified

5.2 Pushing a Branch & Opening a PR
Push your feature branch:
git switch feature/quality-report
git push -u origin feature/quality-report
Desktop Equivalent
Click Push Origin (or Publish Branch if it's new). Then click Create Pull Request — opens GitHub in your browser.
On GitHub:
1 Click "Compare & pull request"
2 Fill in: Title, Description, Reviewers, Labels
3 Click "Create pull request"
Good PR template:
What — Summary of changes
Why — Motivation/context
Testing — How you verified

5.3 Code Review

As a Reviewer:

1. Go to PR's **"Files changed"** tab
2. Review the diff line by line
3. Comment on any line
4. Submit: **Comment**, **Approve**, or **Request**

Review prefixes:

Prefix	Meaning
`nit:`	Non-blocking suggestion
`question:`	Clarification
`blocker:`	Must fix before merge

5.3 Code Review

As a Reviewer:
1. Go to PR's "Files changed" tab
2. Review the diff line by line
3. Comment on any line
4. Submit: Comment, Approve, or Request

Review prefixes:

Prefix	Meaning
nit:	Non-blocking suggestion
question:	Clarification
blocker:	Must fix before merge

5.4 Merging a Pull Request

Method	Description	When to use
Merge commit	Preserves full history	Default
Squash & merge	Combines into one commit	Clean main history
Rebase & merge	Replays on top of main	Linear history

After merging on GitHub:

```
# Click "Delete branch" on GitHub, then locally:
git switch main
git pull
git branch -d feature/quality-report
```

Desktop Equivalent

Desktop auto-detects the merged branch. Click **Fetch/Pull**, then right-click the old branch → Delete.

5.4 Merging a Pull Request
Method | Description | When to use
Merge commit | Preserves full history | Default
Squash & merge | Combines into one commit | Clean main history
Rebase & merge | Replays on top of main | Linear history
After merging on GitHub:
Click "Delete branch" on GitHub, then locally:
git switch main
git pull
git branch -d feature/quality-report
Desktop Equivalent
Desktop auto-detects the merged branch. Click Fetch/Pull, then right-click the old branch → Delete.

5.5 Working with Collaborators

Adding collaborators: Settings → Collaborators → Add people

Role	Can do
Read	View and clone
Triage	Manage issues and PRs (no push)
Write	Push to branches, merge PRs
Maintain	Manage repo settings
Admin	Full access

5.5 Working with Collaborators

Adding collaborators: Settings → Collaborators → Add people

Role	Can do
Read	View and clone
Triage	Manage issues and PRs (no push)
Write	Push to branches, merge PRs
Maintain	Manage repo settings
Admin	Full access

5.6 The Fork & Pull Request Model

For contributing to repos you **don't own**:

1. **Fork** the repository (your copy on GitHub)
2. **Clone** your fork locally
3. Create a **feature branch** and make changes
4. **Push** and open a PR to the original repo
5. Maintainers review and merge

Keeping your fork up to date:

```
git remote add upstream https://github.com/OWNER/REPO.git
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```


5.6 The Fork & Pull Request Model

For contributing to repos you **don't own**:

1. **Fork** the repository (your copy on GitHub)
2. **Clone** your fork locally
3. Create a **feature branch** and make changes
4. **Push** and open a PR to the original repo
5. Maintainers review and merge

Keeping your fork up to date:

```
git remote add upstream https://github.com/OWNER/REPO.git
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

5.7 GitHub Issues & Branch Protection

Issues — built-in task/bug tracking:

- Go to Issues tab → New issue
- Add title, description, labels, assignees

Linking PRs to issues (auto-close on merge):

```
Fixes #12
Closes #12
Resolves #12
```

Protecting `main`**:** Settings → Branches → Protection rules:

- Require a pull request before merging
- Require approvals (1+)
- Require status checks to pass
- Do not allow force pushes

5.7 GitHub Issues & Branch Protection
Issues — built-in task/bug tracking:
Go to Issues tab → New issue
Add title, description, labels, assignees
Linking PRs to issues (auto-close on merge):
Fixes #12
Closes #12
Resolves #12
Protecting main: Settings → Branches → Protection rules:
Require a pull request before merging
Require approvals (1+)
Require status checks to pass
Do not allow force pushes

Exercise 5

Part A — Solo PR Practice (CLI or Desktop):

1. Create a branch, make changes, push to GitHub
2. Open a Pull Request, review “Files changed”
3. Merge the PR, delete the branch, pull locally

Part B — Pair Exercise:

1. Person A creates a repo, adds Person B as collaborator
2. Person B clones (CLI or Desktop), creates branch, opens PR
3. Person A reviews, approves, and merges

Live Demo

Full PR lifecycle — from branch creation to merge, shown in Desktop & CLI.

Exercise 5

Part A — Solo PR Practice (CLI or Desktop):
1. Create a branch, make changes, push to GitHub
2. Open a Pull Request, review “Files changed”
3. Merge the PR, delete the branch, pull locally

Part B — Pair Exercise:
1. Person A creates a repo, adds Person B as collaborator
2. Person B clones (CLI or Desktop), creates branch, opens PR
3. Person A reviews, approves, and merges

Live Demo
Full PR lifecycle — from branch creation to merge, shown in Desktop & CLI.

Advanced Topics & Best Practices

THE UNIVERSITY OF TENNESSEE KNOXVILLE

Part 6: Advanced Topics

Advanced Topics & Best Practices

6.1 Writing Good Commit Messages

Convention:

```
<type>: <short summary in imperative mood>

<optional body - explain WHAT and WHY>

<optional footer - reference issues>
```

Type	Purpose
`feat`	New feature
`fix`	Bug fix
`docs`	Documentation only
`style`	Formatting, no logic change
`refactor`	Code restructuring
`test`	Adding/modifying tests
`chore`	Build process, dependencies

Rules: imperative mood, first line <50 chars, body at 72 chars, reference issues.

6.1 Writing Good Commit Messages
Convention:
<type>: <short summary in imperative mood>
<optional body - explain WHAT and WHY>
<optional footer - reference issues>
Type Purpose
feat New feature
fix Bug fix
docs Documentation only
style Formatting, no logic change
refactor Code restructuring
test Adding/modifying tests
chore Build process, dependencies
Rules: imperative mood, first line <50 chars, body at 72 chars, reference issues.

6.2 Git Stash — Save Work Temporarily

When you need to switch branches but aren't ready to commit:

```
# Save current changes
git stash

# Save with a message
git stash push -m "WIP: login form"

# List stashes
git stash list

# Apply and remove
git stash pop

# Apply (keep in stash)
git stash apply
```

Desktop Equivalent

GitHub Desktop does not support stash directly. Use the terminal for stash, or commit to a WIP branch instead.

6.2 Git Stash — Save Work Temporarily
When you need to switch branches but aren't ready to commit:
Save current changes
git stash
Save with a message
git stash push -m "WIP: login form"
List stashes
git stash list
Apply and remove
git stash pop
Apply (keep in stash)
git stash apply
Desktop Equivalent
GitHub Desktop does not support stash directly. Use the terminal for stash, or commit to a WIP branch instead.

6.4 Finding Files >100 MB Before Pushing

GitHub **rejects** files larger than 100 MB. Find them *before* you commit:

Step 1 — Scan your working directory:

```
# Find files > 100 MB in current folder
find . -type f -size +100M -not -path './.git/*'
```

Step 2 — Auto-add them to .gitignore:

```
# Append every >100MB file to .gitignore
find . -type f -size +100M -not -path './.git/*' \
  | sed 's|^\./||' >> .gitignore
```

Step 3 — Or track them with Git LFS instead:

```
git lfs track "*.h5" "*.parquet" "*.pkl"
git add .gitattributes
```

Pro Tip

Add a pre-commit hook to **automatically block** large files — see the FAQ slides.

6.4 Finding Files >100 MB Before Pushing
GitHub rejects files larger than 100 MB. Find them before you commit:
Step 1 — Scan your working directory:
Find files > 100 MB in current folder
find . -type f -size +100M -not -path './.git/*'
Step 2 — Auto-add them to .gitignore:
Append every >100MB file to .gitignore
find . -type f -size +100M -not -path './.git/*' \
| sed 's|^\./||' >> .gitignore
Step 3 — Or track them with Git LFS instead:
git lfs track "*.h5" "*.parquet" "*.pkl"
git add .gitattributes
Pro Tip
Add a pre-commit hook to automatically block large files — see the FAQ slides.

6.5 Already Committed a Large File? Here's the Fix

If you committed a >100 MB file and `git push` fails:

Option A — File is in the last commit (not yet pushed):

```
# Remove from staging, keep on disk
git rm --cached large_dataset.h5
echo "large_dataset.h5" >> .gitignore
git commit --amend --no-edit
```

Option B — File is in older commits (rewrite history):

```
# One-time install
pip install git-filter-repo
git filter-repo --path large_dataset.h5 --invert-paths
git remote add origin <url>
git push --force --set-upstream origin main
```

⚠ Warning: Rewriting history changes commit hashes. Coordinate with your team before force-pushing.

6.5 Already Committed a Large File? Here's the Fix

If you committed a >100 MB file and `git push` fails:

Option A — File is in the last commit (not yet pushed):

```
# Remove from staging, keep on disk
git rm --cached large_dataset.h5
echo "large_dataset.h5" >> .gitignore
git commit --amend --no-edit
```

Option B — File is in older commits (rewrite history):

```
# One-time install
pip install git-filter-repo
git filter-repo --path large_dataset.h5 --invert-paths
git remote add origin <url>
git push --force --set-upstream origin main
```

⚠ Warning: Rewriting history changes commit hashes. Coordinate with your team before force-pushing.

6.10 Common Mistakes & How to Fix Them

"Committed to the wrong branch!"

```
git reset --soft HEAD~1          # undo commit, keep changes
git switch correct-branch
git commit -am "Your message"
```

"Need to undo a pushed commit!"

```
git revert <commit-hash>         # safe: creates undo commit
git push
```

"Accidentally deleted a file!"

```
git restore deleted_file.txt
```

"Repo has secrets!"

```
git rm --cached secrets.env
echo "secrets.env" >> .gitignore
git commit -am "Remove secrets from tracking"
```


6.10 Common Mistakes & How to Fix Them
"Committed to the wrong branch!"
git reset --soft HEAD~1 # undo commit, keep changes
git switch correct-branch
git commit -am "Your message"
"Need to undo a pushed commit!"
git revert <commit-hash> # safe: creates undo commit
git push
"Accidentally deleted a file!"
git restore deleted_file.txt
"Repo has secrets!"
git rm --cached secrets.env
echo "secrets.env" >> .gitignore
git commit -am "Remove secrets from tracking"

6.11 Best Practices Summary

Practice	Why
Commit often	Small commits are easier to understand/revert
Clear commit messages	Future you will thank present you
Use branches	Keep `main` stable
Pull before push	Minimize merge conflicts
Use `.gitignore`	Don't track generated/secret files
Review PRs carefully	Catch bugs and share knowledge
Tag releases	Mark important milestones
Never commit passwords	API keys, passwords
Use SSH keys	Now Enforced
Push regularly	Back up with remotes

6.11 Best Practices Summary

Practice	Why
Commit often	Small commits are easier to understand/revert
Clear commit messages	Future you will thank present you
Use branches	Keep `main` stable
Pull before push	Minimize merge conflicts
Use `.gitignore`	Don't track generated/secret files
Review PRs carefully	Catch bugs and share knowledge
Tag releases	Mark important milestones
Never commit passwords	API keys, passwords
Use SSH keys	Now Enforced
Push regularly	Back up with remotes

Frequently Asked Questions

Quick tutorials for common problems

FAQ
Frequently Asked Questions
Quick tutorials for common problems

FAQ 1: “Push rejected — file exceeds 100 MB”

Symptom: `remote: error: File X is 150.00 MB; this exceeds GitHub’s limit of 100.00 MB`

Quick fix (file in last commit):

```
git rm --cached big_file.csv
echo "big_file.csv" >> .gitignore
git commit --amend --no-edit
git push
```

If the file is several commits back:

```
pip install git-filter-repo
git filter-repo --path big_file.csv --invert-paths
git remote add origin <url>
git push --force
```

Prevention: add a pre-commit hook (see FAQ 5).

FAQ 1: “Push rejected — file exceeds 100 MB”
Symptom: remote: error: File X is 150.00 MB; this exceeds GitHub’s limit of 100.00 MB
Quick fix (file in last commit):
git rm --cached big_file.csv
echo "big_file.csv" >> .gitignore
git commit --amend --no-edit
git push
If the file is several commits back:
pip install git-filter-repo
git filter-repo --path big_file.csv --invert-paths
git remote add origin <url>
git push --force
Prevention: add a pre-commit hook (see FAQ 5).

FAQ 2: “I want to undo my last commit”

Situation	Command	Effect
Undo commit, keep staged	`git reset -soft HEAD~1`	Safe
Undo commit, unstage	`git reset HEAD~1`	Safe
Undo & discard changes	`git reset -hard HEAD~1`	**Destructive!**
Already pushed?	`git revert HEAD`	Creates undo commit

```
# Most common: undo commit but keep your work
git reset --soft HEAD~1
# Now edit, re-stage, and commit again
```

Desktop Equivalent

History tab: right-click the commit.
Use **Undo Commit** (last) or **Revert Commit** (if pushed).

FAQ 2: “I want to undo my last commit”
Desktop Equivalent
History tab: right-click the commit.
Use **Undo Commit** (last) or **Revert Commit** (if pushed).

FAQ 3: “I committed to main instead of a branch!”

```
# Step 1: Create the branch you meant to use
#         (keeps your commits)
git branch feature/my-work

# Step 2: Reset main back to before your commits
git reset --hard HEAD~2   # undo last 2 commits on main

# Step 3: Switch to your branch
git switch feature/my-work
# Your commits are safely here!
```

⚠ If you already pushed to main

```
git revert HEAD~2..HEAD     # create undo commits
git push                     # push the reverts
# then cherry-pick or redo on the correct branch
```


FAQ 3: “I committed to main instead of a branch!”
Step 1: Create the branch you meant to use
(keeps your commits)
git branch feature/my-work
Step 2: Reset main back to before your commits
git reset --hard HEAD~2 # undo last 2 commits on main
Step 3: Switch to your branch
git switch feature/my-work
Your commits are safely here!
If you already pushed to main
git revert HEAD~2..HEAD # create undo commits
git push # push the reverts
then cherry-pick or redo on the correct branch

FAQ 4: “Detached HEAD state” — What?

This happens when you check out a specific commit instead of a branch.

```
# You did something like:
git checkout abc1234    # now in detached HEAD

# Fix: create a branch from here
git switch -c rescue-branch

# Or just go back to main:
git switch main
```

Rule of thumb: Always work on a **branch**, not a raw commit.

Desktop Equivalent

Desktop prevents this situation — it always keeps you on a named branch.

FAQ 4: “Detached HEAD state” — What?
This happens when you check out a specific commit instead of a branch.
You did something like:
git checkout abc1234 # now in detached HEAD
Fix: create a branch from here
git switch -c rescue-branch
Or just go back to main:
git switch main
Rule of thumb: Always work on a branch, not a raw commit.
Desktop Equivalent
Desktop prevents this situation — it always keeps you on a named branch.

FAQ 6: “.gitignore isn’t ignoring my files!”

`.gitignore` only works on **untracked** files. If Git already tracks a file:

```
# Step 1: Remove from tracking (keep file on disk)
git rm --cached data/results.csv

# Step 2: Add to .gitignore
echo "data/results.csv" >> .gitignore

# Step 3: Commit
git add .gitignore
git commit -m "chore: stop tracking results.csv"
```

To untrack *everything* and re-apply .gitignore:

```
git rm -r --cached .
git add .
git commit -m "chore: re-apply .gitignore"
```

This is harmless — no files are deleted from your disk.

FAQ 6: “.gitignore isn’t ignoring my files!”
.gitignore only works on untracked files. If Git already tracks a file:
Step 1: Remove from tracking (keep file on disk)
git rm --cached data/results.csv
Step 2: Add to .gitignore
echo "data/results.csv" >> .gitignore
Step 3: Commit
git add .gitignore
git commit -m "chore: stop tracking results.csv"
To untrack everything and re-apply .gitignore:
git rm -r --cached .
git add .
git commit -m "chore: re-apply .gitignore"
This is harmless — no files are deleted from your disk.

FAQ 7: “Should I merge or rebase?”

	Merge	Rebase
Command	`git merge feature`	`git rebase main`
History	Preserves branches	Linear (cleaner)
Safety	Never rewrites history	Rewrites commits
Use when	Shared branches, PRs	Local cleanup before PR

```
# Rebase your feature onto latest main
git switch feature/demand-forecast
git rebase main
# Then push (force if already pushed)
git push --force-with-lease
```

Rule for Beginners

When in doubt, use `merge`. It’s always safe.

FAQ 7: “Should I merge or rebase?”

	Merge	Rebase
Command	`git merge feature`	`git rebase main`
History	Preserves branches	Linear (cleaner)
Safety	Never rewrites history	Rewrites commits
Use when	Shared branches, PRs	Local cleanup before PR

```
# Rebase your feature onto latest main
git switch feature/demand-forecast
git rebase main
# Then push (force if already pushed)
git push --force-with-lease
```

Rule for Beginners

When in doubt, use merge. It’s always safe.

FAQ 8: “Can I just download the ZIP?”

Yes, but you lose everything that makes Git useful:

Feature	Clone	ZIP Download
Full history	✓	×
Pull future updates	✓	×
Push your changes	✓	×
Switch branches	✓	×
Small download	×	✓

Always clone if you plan to contribute or stay updated:

```
git clone git@github.com:USER/REPO.git
```

Desktop Equivalent

File → Clone Repository → paste URL.

FAQ 8: “Can I just download the ZIP?”
Yes, but you lose everything that makes Git useful:
Always clone if you plan to contribute or stay updated:
Desktop Equivalent
File → Clone Repository → paste URL.

Git Cheat Sheet (1/2)

Setup: `git config --global user.name "Name"`

Create & Clone:

```
git init        git clone <url>
```

Basic Workflow:

```
git status      git add <file>
git add .       git commit -m "msg"
```

View Changes & History:

```
git diff                    git diff --staged
git log --oneline           git log --oneline --graph --all
```

Branching:

```
git branch              git switch -c <name>
git merge <branch>      git branch -d <name>
```


Git Cheat Sheet (1/2)

Git Cheat Sheet (2/2)

Remotes:

```
git remote add origin <url>    git push -u origin main
git push                       git pull
```

Undoing:

```
git restore <file>               git restore --staged <file>
git commit --amend               git reset --soft HEAD~1
```

Stash & Tags:

```
git stash                    git stash pop
git tag -a v1.0.0 -m "msg"    git push origin --tags
```

Complete Workflow:

```
git switch -c feature/work    # 1. Branch
git add . && git commit -m "msg"  # 2. Commit
git push -u origin feature/...    # 3. Push
```


Git Cheat Sheet (2/2)

Remotes:

git remote add origin <url> git push -u origin main
git push git pull

Undoing:

git restore <file> git restore --staged <file>
git commit --amend git reset --soft HEAD~1

Stash & Tags:

git stash git stash pop
git tag -a v1.0.0 -m "msg" git push origin --tags

Complete Workflow:

git switch -c feature/work # 1. Branch
git add . && git commit -m "msg" # 2. Commit
git push -u origin feature/... # 3. Push

Where to Go Next

- **Git Documentation:** `https://git-scm.com/doc`
- **GitHub Skills:** `https://skills.github.com` — interactive courses
- **Pro Git Book (free):** `https://git-scm.com/book/en/v2`
- **GitHub Desktop Docs:** `https://docs.github.com/en/desktop`
- **Learn Git Branching:** `https://learngitbranching.js.org`
- **GitHub Docs:** `https://docs.github.com`

Where to Go Next

- Git Documentation: https://git-scm.com/doc
- GitHub Skills: https://skills.github.com — interactive courses
- Pro Git Book (free): https://git-scm.com/book/en/v2
- GitHub Desktop Docs: https://docs.github.com/en/desktop
- Learn Git Branching: https://learngitbranching.js.org
- GitHub Docs: https://docs.github.com

Questions?

Thank you!

Jose Tupayachi
Department of Industrial & Systems Engineering — UTK

Tutorial materials available at:
`https://github.com/jtupayachi/github_tutorial`

Questions?
Thank you!
Jose Tupayachi
Department of Industrial & Systems Engineering — UTK
Tutorial materials available at:
https://github.com/jtupayachi/github_tutorial